Project 1 Report: Audio Classification Using Edge Impulse on Arduino Nano 33 BLE Sense

Hannes Kullman: 2003002; Zakaria Bouzada: 2102782

1. Introduction

This project focused on developing a lightweight speech classification system capable of recognizing city names in spoken queries such as "What is the weather in ...?" using the Arduino Nano 33 BLE Sense board. We used the Edge Impulse platform to collect audio samples, engineer features, train a neural network, and deploy the trained model to an embedded system. The primary goal was to accurately identify keywords under various real-world conditions. In our project we used the cities Helsinki and Tallinn.

---

2. Performed Tasks

2.1 Project Setup

A shared account was created on Edge Impulse, which is a platform which was well suited for the task of this project. It supplied a user-friendly, clear and straightforward way to perform the steps taken. Following the instructions on https://docs.edgeimpulse.com/docs/edge-ai-hardware/mcu/arduino-nano-33-ble-sense the necessary dependencies were installed. The Edge Impulse firmware for Arduino Nano 33 BLE Sense was necessary for recording audio samples on the Arduino for the Edge Impulse platform. In conjunction with this it was necessary to install the Edge Impulse CLI which provided the commands for providing communication between the Arduino and laptop. By connecting the Arduino through USB-Micro with the laptop and pressing the reset button (of the Arduino) twice, it restarted into bootloader. At this point it was possible to flash the firmware onto the board by running the flash_windows.bat file provided by the Edge Impulse firmware download. By running the edge-impulse-daemon command (from the CLI) it was possible to sign into Edge Impulse through the Arduino and thus it showed up as “connected” on the Edge Impulse website.

2.2 Dataset Collection

By following the tutorial at https://docs.edgeimpulse.com/docs/tutorials/end-to-end-tutorials/audio/responding-to-your-voice we were able to record audio samples of the phrases "What's the weather in Helsinki?" and '"What's the weather in Tallinn?". The recording was done by three different people, all of whom spoke to the Arduino microphone. Voice1 was used to record Helsinki and Tallinn 16 times respectively. This was also the case for Voice2. The third voice was used to record each city twelve times. This adds up to 88 recorded samples. During some of the samples there was intentional noise in the background. About 20 samples were affected by this to give the samples some greater deviation from each other. Some samples were unintentionally cut off at the city name, which might have negatively impacted the training of the neural network model. We used a sample length of 2100 milliseconds. Within ten-second windows we were thus able to record four clearly separated samples. All the samples were divided into two classes "Helsinki" and "Tallin" of similar sizes. However, the number of samples turned out rather limited during training.

2.3 Feature Extraction

Following the instructions on Moodle for session 2 we split the dataset into test/train datasets. This split gave a 27% share for the test set. Default values were first used for creating the impulse (window size 1000ms) and adding a MFCC block as well as a classification block. This step is peculiarly simple. At this point it was possible to generate features which were visually represented on a graph as in Figure 1.

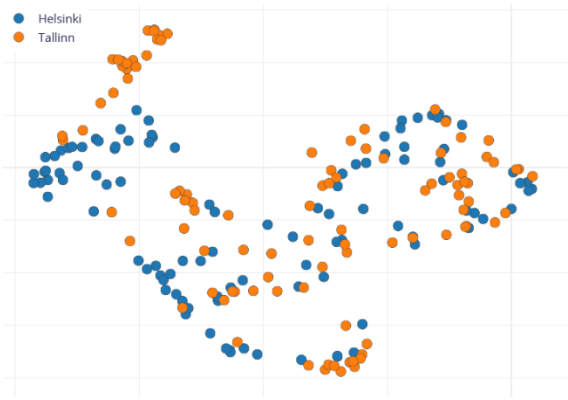

Figure 1: The initial feature spread when window size was set to 1000ms.

However, there was little difference between the Helsinki and Tallinn features. By playing the audio of one of the samples it was possible to notice that the window size had effectively divided the samples into two halves. Since the classification of a sample is only determined by the last word of its sentence it is not possible to classify a sentence based on the first half, but this was that the window size setting had led to. For this reason, the window size was increased to 2100ms to cover the entire sample size, resulting in a more distributed and separable distribution of features as shown in Figure 2.

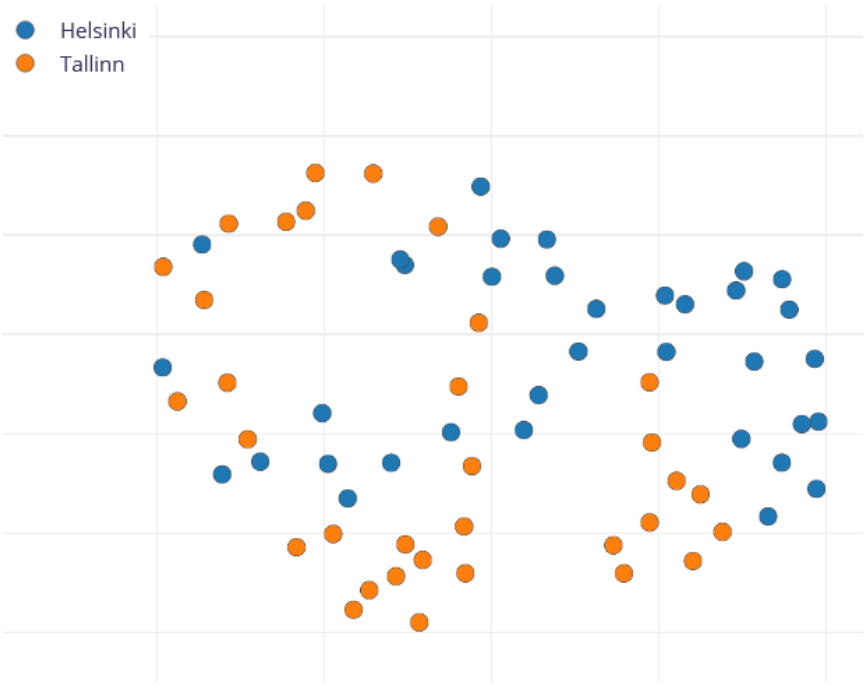

Figure 2: The spread of features with window size set to 2100ms.

2.4 Model Training

With the improved features it became possible to move to the training of the neural network model. It was possible to tweak the training process (and also the model) through the graphical user interface of Edge Impulse. A couple of different configurations were tested. By training the model with the default configuration it was possible to achieve a 78.6% accuracy (Figure 3).

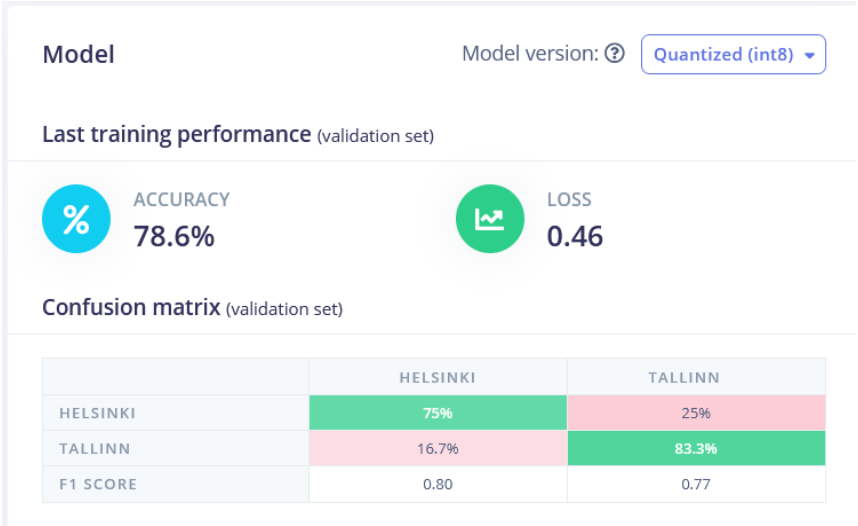

Figure 3: Training results of the default configuration.

In the second run, data augmentation with default settings was tested (Figure 4). This had the same results.

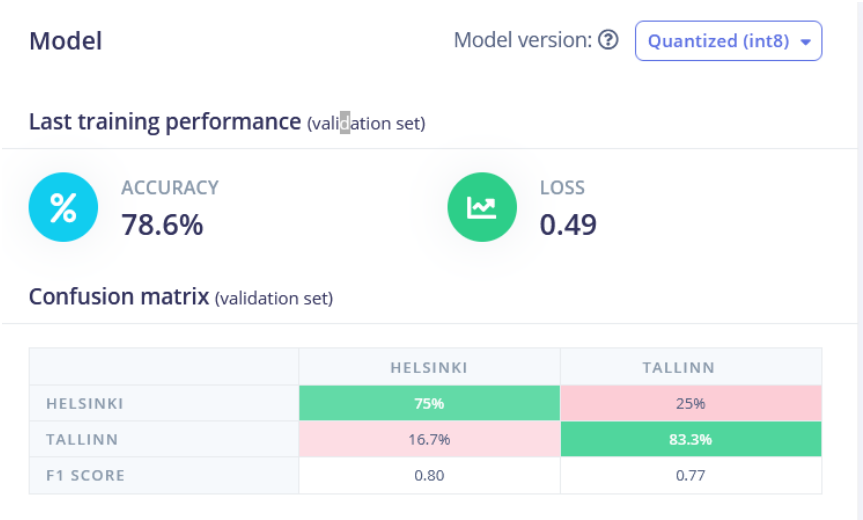

Figure 4: Training results with default data augmentation.

It was also possible to alter a variable called “Learning rate” with a default value of 0.005. This value was by trial-and-error found to work best at 0.02. This configuration together with data augmentation resulted in a training accuracy of 85.7% (Figure 5). A value of 0.1 on the learning rate reduced the accuracy below 50%.

Last training performance (validation set)

ACCURACY
85.7%

LOSS
0.32

Confusion matrix (validation set)

	HELSINKI	TALLINN
HELSINKI	87.5%	12.5%
TALLINN	16.7%	83.3%
F1 SCORE	0.88	0.83

Figure 5: Training results with learning rate set to 0.2

Attempts were made to improve accuracy by using the learned optimizer, but this led to the previously mentioned 78.6% accuracy. At this point we realized that the reason this same accuracy percentage was achieved by several models must be because of the limited number of training samples. Finally, an improved training accuracy was achieved by removing data augmentation and only setting learning rate to 0.02 (no other changes from default configuration). This achieved a training accuracy of 92.9% (Figure 6). This jump (92.9-85.7=7.2) is just as long in percentiles as the initial improvement (85.6-78.6=7.0). This was a further indication of the impacts of a small training set. Because of this the improved (92.9%) accuracy was assumed to be due to overfitting as the learning rate should be decreased if the model appears to overfit. In other words, we chose to use the model trained with a learning rate of 0.2 coupled with data augmentation.

Model Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY
92.9%

LOSS
0.20

Confusion matrix (validation set)

	HELSINKI	TALLINN
HELSINKI	87.5%	12.5%
TALLINN	0%	100%
F1 SCORE	0.93	0.92

Figure 6: Training results with learning rate set to 0.2 with no other changes.

2.5 Model Deployment

To make the trained model useful it was exported from Edge Impulse as an Arduino Library. Including this library in Arduino was simple and straightforward. It was later possible to open the model directly with a ready-made program for audio recording and inference of the model. The nano_BLE33_sense_microphone_continuous program was difficult to use, as it was difficult to know at what point the program started to record audio. The second program (nano_BLE33_sense_microphone) was more clear about this and because of this we present our results based on this program.

3. Results

The results were obtained by running the nano_BLE33_sense_microphone program. By speaking the sentences at the indicated time (when the program said “recording”) it was possible for the program to classify the spoken sentence. The same people (Voice1, Voice2 and Voice3) spoke their sentences, normally, 25 times for each city. The results are presented in Table 1.

Table 1. The number of correct classifications out of the spoken sentences for each person for each city.

Person City	Helsinki	Tallinn
Voice1	23/25	16/25
Voice2	21/25	15/25
Voice3	25/25	3/25

The classification for Helsinki worked better than the classification for Tallinn. This is rather surprising. While evaluating the results we referred to Figure 2 and noticed that two of the samples at the left, although different classes were placed in similar locations of the graph even though we perceived them as distinct. In fact both sentences were almost exemplary versions of the supposed sentences, but none the less they were located close to each other. Together with

the fact that the classification of Helsinki appeared to perform better, this might indicate that the model is not classifying a sample simply based on the last word (which is the one a human would refer to), but rather it might rely on the tone of voice or other indication. This hypothesis can be supported by the fact that Voice3 was almost unable to be recognized as saying a Tallinn sentence even though this was done 25 times. This would suggest that the model recognized Voice3 as somehow connected to the Helsinki class. We also tested the model with silent input, that is, no one was speaking while recording those test inputs. Out of 50 samples recorded by the model, 24 were classified as Helsinki. There was a worry that the model might recognize nonsense sentences as Tallinn because of its lower accuracy, but the results for the Tallinn classification is higher than 12/25 which seems to contradict that worry.

```
18:36:54.952 -> Predictions (DSP: 146 ms., Classification: 11 ms., Anomaly: 0 ms.):
18:36:54.952 ->     Helsinki: 0.86719
18:36:54.952 ->     Tallinn: 0.13281
18:36:57.068 -> Predictions (DSP: 146 ms., Classification: 11 ms., Anomaly: 0 ms.):
18:36:57.068 ->     Helsinki: 0.18750
18:36:57.068 ->     Tallinn: 0.81250
18:36:59.169 -> Predictions (DSP: 146 ms., Classification: 11 ms., Anomaly: 0 ms.):
18:36:59.169 ->     Helsinki: 0.35547
18:36:59.169 ->     Tallinn: 0.64453
```

Figure A. Running of the first program, which was not used for the results.

```
19:29:49.609 -> Starting inferencing in 2 seconds...
19:29:51.576 -> Recording...
19:29:53.651 -> Recording done
19:29:53.963 -> Predictions (DSP: 279 ms., Classification: 11 ms., Anomaly: 0 ms.):
19:29:53.963 ->     Helsinki: 0.42188
19:29:53.963 ->     Tallinn: 0.57812
19:29:53.963 -> Starting inferencing in 2 seconds...
19:29:55.965 -> Recording...
19:29:58.030 -> Recording done
19:29:58.279 -> Predictions (DSP: 279 ms., Classification: 11 ms., Anomaly: 0 ms.):
19:29:58.279 ->     Helsinki: 0.82031
19:29:58.279 ->     Tallinn: 0.17969
```

Figure B. Running of the second program (nano_BLE33_sense_microphone) which was used to check the results. It was necessary to start talking exactly when “Recording…” showed up, otherwise it was possible the recording would break at the keyword.

4. Conclusions and Comments

Through this project, we built a binary keyword spotting system using Edge Impulse and deployed it to an embedded Arduino Nano device. The results suggest that it is possible to achieve reasonable speech classification with limited hardware resources and audio samples. Variations in voices and environmental noise were supposed to enhance model robustness and might have achieved this to some extent. None the less, our model had some flaws, particularly in relation to Voice3. The classification of Tallinn could have been better. Whereas the tools worked smoothly our greatest limitation was the small training set. Through this work it has become clear that it is necessary to have larger sample sets for achieving accurate training and inference of neural networks for speech recognition. Besides enlarging the number of recorded audio samples, future extensions could include greater variation of speech and noise, include support for additional cities and unclassified sentences, and integration with external APIs.

1. Have you learned anything new?
   This was the first time we did machine learning with audio data. Through our mistakes we have learned (or at least gathered some ideas) how you can collect, handle and train using this kind of data. Since audio exists in many variations it seems necessary to include many variations in the training data. It is also impossible to be careful with the samples themselves. If key areas of the recording are noisy or broken, this might confuse the model unnecessarily. This might actually be a cause of the issue with Tallinn.
2. Did anything surprise you?
   The tools were actually surprisingly well-behaved, and the instructions were clear. When Voice3 was testing Helsinki it was very surprising to see a 25/25 row. We jokingly said that now there will be no matches for Tallinn, which turned out to essentially be the case. This too was surprising and also interesting.
3. Did you find anything challenging?
   Getting started with the audio recording was difficult since we were living in different places, and sitting in a room with not-so-well-isolated walls and repeatedly saying

“What’s the weather in Helsinki?” was difficult not to procrastinate. Understanding certain flaws of the final model was also a challenge.

4. Did you find anything satisfying?

   The GUI of Edge Impulse. Also, the model worked fairly well even though there were few training samples.